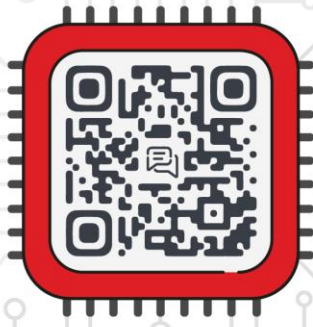




نظري

15

4

2250 sp

كلية الهندسة المعلوماتية

السنة الرابعة

توصيف المتطلبات الغير وظيفية المتقدمة

د . عمار جوخدار

محتوى مجاني غير مخصص للبيع التجاري



RB Informatcs 2026/4/20

برمجة تفرعية

في هذه المحاضرة سنكمل رحلة توصيف المتطلبات غير الوظيفية لنكون قد انتهينا من الإجابة على سؤال "ما هي المشكلة"، لننتقل مباشرة في النصف الثاني من المحاضرة للإجابة على سؤال "كيف يتم حل المشكلة" ونبدأ رحلة التعرف على الحلول.

كنا قد وصلنا في المحاضرة السابقة إلى أنواع إدارة المناقلات، وسنكمل من بعدها:

7. التصاعديّة Scalability (قابليّة التوسّع) :

ولها نوعين:

1. التصاعديّة الوظيفية (Functional Scalability):

وهي القدرة على إضافة وظائف وخدمات إضافية على البرنامج دون الاضطرار إلى تعديل الكود القديم، أي دون الاضطرار لتغيير شيء سبق برمجته.

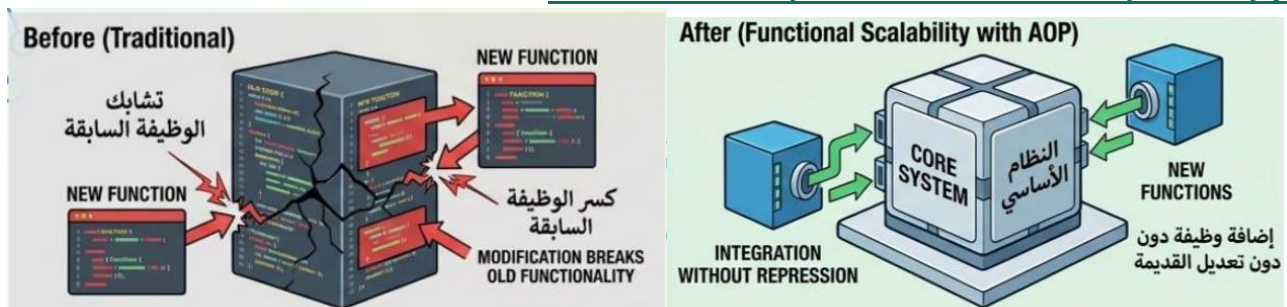
■ نقول عن برنامج أنه قابل للتوسّع عندما يمكننا إضافة وظائف جديدة دون التأثير على الوظائف السابقة، بالتالي تصبح كل وظيفة بحد ذاتها قابلة للبرمجة من قبل أحد المبرمجين لتتكامل هذه الوظائف سوياً دون تراجع ودون تدخل بشري.

- تعد التصاعديّة من أهم الخصائص غير الوظيفية لأي برنامج.
- قامت التصاعديّة بتفادي عدة مشاكل ممكنة الحدوث:

قد يؤدي تعديل أو تطوير برنامج موجود إلى تراجع عن وظيفة كانت محققة مسبقاً، أي تؤثر الإضافات الجديدة على الوظائف الموجودة مسبقاً، وقد يكون زمن التعديل على برنامج معين مساوياً لزمن تطوير هذا البرنامج من الصفر لأنه غير قابل للتوسّع .

◀ حيث قامت التصاعديّة الوظيفية بحل هذه المشاكل.

أهم وأفضل طريقة لتطبيق التصاعديّة الوظيفية هي AOP.



2. التصاعديّة من حيثُ المُستخدمين (User Scalability):

وهي القدرة على تحمّل عدد متزايد من المُستخدمين مع مرور الوقت ، والتعامل مع أعداد كبيرة جداً منهم دون الحاجة لإجراء أي تغيير أو تعديل في الكود البرمجي للنظام، فقط نكتفي بإضافة عتاديات جديدة Hardware دون تغيير العتاديات الحالية أيضاً.

▪ متى نقول عن برنامج أنه قابل للتوسّع من ناحية المُستخدمين؟

عندما يكون قابلاً لتحمل أعداد أكبر وأكبر من المُستخدمين مهما بلغ عددهم دون الحاجة لتعديل كود أو تغيير عتاديات الحالية، بل فقط يمكن إضافة عتاديات جديدة.

لأن إضافة عتاديات (Hardware) يُعتبر رخيص مقارنةً بالبرمجة التي تُعدّ مُكلفة.

مثال: يمكن إضافة سيرفرات (خوادم) جديدة على النظام حيث يتم توزيع الحمل عليهم، حيث تُكفّر إضافة سيرفرات أقل بكثير من تكلفة تعديل الكود، بشرط أن يكون النظام قابلاً للاستفادة من عدة سيرفرات (أي يُحقق مفهوم Load Balancing).

ملاحظة:

- عملية توزيع الحمل Load Balancing هي عملية برمجية بشكل أساسي وليست عتادية حيث إضافة السيرفرات تُعدّ وسيلة لتحقيق التوسعية الأفقية (Horizontal Scaling) لدعم تحمّل عدد أكبر من الطلبات أي زيادة القدرة على تحمّل الضغط.
- في الأنظمة البرمجية التي تعتمد على المُعالجة المتوازية (Parallel Processing)، يمكن تحقيق درجة عالية من التصاعديّة (Scalability)، لأن النظام يكون قادراً على توزيع المهام على عدد كبير من العمليات أو الخيوط البرمجية القابلة للتهيئة والإعداد (Configuration) ويتم ذلك غالباً ضمن طبقة منطق العمل الوسطى (Business Logic Layer) في البنية ثلاثية الطبقات (3-Tier Architecture).
- لكن العمل المتوازي لا يعني فقط تشغيل عدد كبير من المُستخدمين في الوقت نفسه، بل يتطلب أيضاً إدارة دقيقة لاستهلاك الموارد مثل المعالج (CPU) والذاكرة (Memory) لذلك يجب على المصمم تحديد حصص وحدود معينة لكل مُستخدم لمنع استهلاك الموارد بشكل مُفرط قد يؤثر على أداء النظام بالكامل.
- **ومن الأمثلة على ذلك:**
 - تحديد حجم معيّن للتقارير أو كمية البيانات التي يستطيع المُستخدم طلبها، وعدم السماح بتحميل كميات ضخمة دفعة واحدة، وإنما تقسيم البيانات وإرسال جزء محدد منها في كُل طلب، مما يساهم في تحسين الأداء وتحقيق العدالة بين المُستخدمين.

من أهم الآليات المُستخدمة لتحقيق Scalability في الأنظمة البرمجية : Load Balancing and Clustering سنتحدث عنهم بالتفصيل .

هناك عدة أدوات خاصة لاختبار توسع البرنامج من حيث المستخدمين وهي أدوات خاصة بالـ Bench marking ومحاكاة عدة مستخدمين مثل: JMeter, Open STA, Web Runner .
وبعضها مجاني كما يوجد غيرها ,حيث تقوم هذه الأدوات بمحاكاة عدد المستخدمين.

8. إدارة الموارد (Resource Management):

يُقصد بها استهلاك أقل ما يمكن من الموارد لكل مُستخدم وعدم تجاوز الموارد المُتاحة، لذلك من الضروري معرفة كل طلب Request يأتي للنظام كم سيستهلك من الموارد (Memory - CPU - Bandwidth - Hard Disk) للتمكن من تحديد عدد المُستخدمين المتنافسين (الذين يستخدمون النظام على التوازي) الممكن للنظام أن يخدمهم.

- كيف يُمكن تحديد عدد المُستخدمين الذين يستطيع النظام تخديمهم :

في الواقع لا يعتمد ذلك على عدد المُستخدمين فقط , بل يعتمد أيضاً على نوع العمليات التي ينفّذها كل مُستخدم ومقدار الموارد التي تستهلكها هذه العمليات من المعالج والذاكرة .

فإذا كان كل مُستخدم ينفّذ طلبات مختلفة تماماً من حيث الحجم والتعقيد , فلن يكون لعدد المستخدمين وحده معنى دقيق في قياس أداء النظام وقابليته للتصاعديّة (Scalability) لذلك تحرص الأنظمة القابلة للتوسع على أن تكون الطلبات (Requests) مُتقاربة قدر الإمكان من حيث استهلاك الموارد , أو يتم فرض حدود معيّنة على حجم البيانات والعمليات المسموح بها لكل طلب .

ومن الأمثلة على ذلك:

تحديد عدد السجلات التي يُمكن إرجاعها في التقرير الواحد، أو تقسيم البيانات الكبيرة إلى دفعات صغيرة (Lazy Loading) , وذلك بهدف منع استهلاك الذاكرة والمعالج بشكل مُفرط، وضمان استقرار أداء النظام لجميع المستخدمين.

■ آليات إدارة الموارد (Resource Management Mechanisms):

هناك عدة طرق وتقنيات لتقليل وإدارة استهلاك الموارد:

■ (1) Lazy List:

تقوم بتقسيم الطلبات Requests الكبيرة إلى طلبات أصغر.

مثال:

نتائج البحث في Google لا يتم عرضها جميعاً بل يتم عرض كل جزء على حدى.

في نظام بنك يحوي عدداً كبيراً جداً من العملاء، عند طلب طباعة أسماء جميع العملاء سيستهلك وقتاً كبيراً والنظام Online فلا نسمح له بذلك , أو من الممكن إضافة مخدّم Server جانبي و Backup Data Base للقيام بمثل هذه المهمات.

■ (2) Caching:

تحويل الضغط عن الـ CPU إلى ضغط على الذاكرة وذلك لتخزين المعلومات التي نحتاجها بشكل دائم والعكس.

تستخدم بعض التطبيقات المؤسساتية الـ Caching، والتي تعتمد على تخزين البيانات المتكررة أو المُستخدمة بشكل كبير داخل الذاكرة، بحيث يتمكن جميع المُستخدمين من الاستفادة منها دون الحاجة إلى إعادة تحميلها أو إعادة تنفيذ العمليات نفسها في كل مرة.

وتحقق هذه الآلية نوعاً من الـ Sharing للبيانات بين المُستخدمين، مما يخفف الضغط على موارد النظام مثل المعالج (CPU) والذاكرة (RAM)، ويساهم في تحسين الأداء وتسريع الاستجابة.

■ (3) Thread Pooling:

هي آلية تُستخدم في الأنظمة المتوازية لإدارة الـ Threads بشكل منظم وفعال. بدلاً من إنشاء Thread جديد لكل طلب من المُستخدمين، يقوم النظام بإنشاء مجموعة جاهزة مُسبقاً من الـ Threads تسمى Pool، ثم يتم توزيع الطلبات عليها عند الحاجة.

الفائدة من ذلك :

- تخفيف استهلاك المُعالج والذاكرة الناتج عن إنشاء Threads جديدة باستمرار.
- تحسين الأداء وزمن الاستجابة.
- التحكم بعدد العمليات المتوازية ومنع النظام من الانهيار عند زيادة الطلبات.
- إعادة استخدام Threads موجودة بدلاً من إنشاء Threads جديدة.

فعلي سبيل المثال :

إذا كان لدينا Thread Pool بحجم 50 فهذا يعني أن النظام يسمح بتنفيذ 50 طلباً بشكل متوازي ، بينما تنتظر الطلبات الزائدة في Queue حتى يُصبح أحد الـ Threads متاحاً .

■ (4) Session:

هي آلية تُستخدم للاحتفاظ بمعلومات خاصة بكل مستخدم أثناء عمله على النظام. والفكرة الأساسية منها هي تخصيص مساحة محددة وثابتة تقريباً من الذاكرة لكل مُستخدم، بحيث يحتفظ النظام بالمعلومات الضرورية المُتعلقة به بين الـ Requests المختلفة .

وتتضمن هذه المعلومات عادةً:

- هوية المستخدم.
- الصلاحيات.
- الإعدادات الخاصة به.
- البيانات المؤقتة المرتبطة بجلسة العمل.



ويتم ربط كل مُستخدم بـ Session خاصة به عبر Session ID يُرسل مع كل Request ومن المهم في الأنظمة القابلة للتوسع (Scalable Systems) أن يكون حجم البيانات المخزنة داخل الـ Session محدوداً وثابتاً تقريباً لكل مُستخدم، لأن زيادة حجم الـ Session تؤدي إلى زيادة استهلاك الذاكرة وانخفاض قدرة النظام على دعم عدد أكبر من المستخدمين.

وفي بعض الحالات يتم تخفيف الحمل عن السيرفر عبر استخدام Cookies ، وهي ملفات صغيرة تُخزن داخل متصفح المستخدم على جهازه الخاص، وتُستخدم للاحتفاظ ببعض البيانات البسيطة أو معلومات التعريف بالمستخدم بدلاً من تخزين كل شيء داخل الـ Session على السيرفر.

ملاحظة:

هذه الصعوبات لم تكن واضحة في نموذج Client/Server التقليدي، لأن عدد المستخدمين كان محدوداً وبيئة العمل مغلقة نسبياً، أما تطبيقات الويب (Web Based) فهي مفتوحة لعدد كبير وغير معروف من المستخدمين يعملون بشكل متزامن، لذلك تظهر مشاكل التوسعية (Scalability) وإدارة الموارد وتوزيع الحمل بشكل أكبر.

والآن لنفكر قليلاً كيف يمكن للبرنامج أن يعمل على عدة سيرفرات؟ عن طريق

9. Clustering, Load Balancing and Fault Tolerance (العُنقدة، موازنة الحمل، وتحمل الأعطال):

تفيد في تقسيم الطلبات بين مجموعة من المخدمات وتُميّز الحالات التالية:

■ موازنة الحمل (Load Balancing):

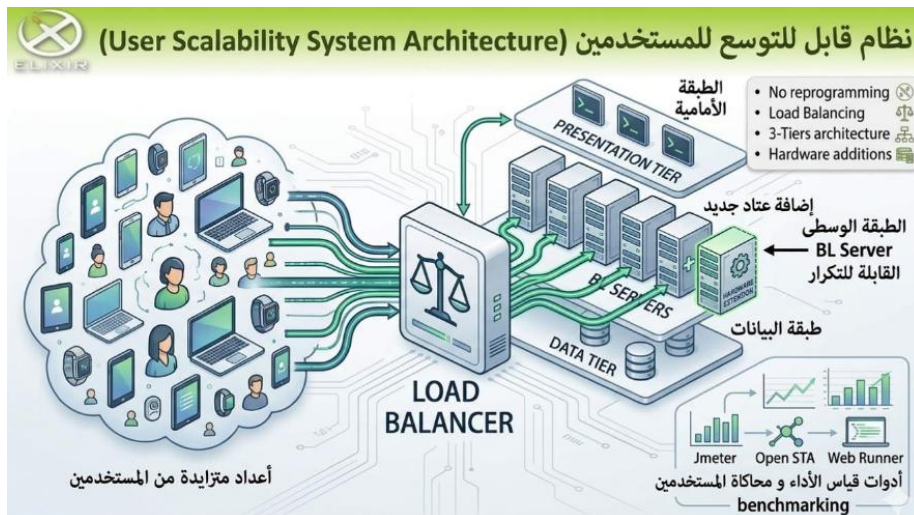
تعتمد فكرة توزيع الحمل (Load Balancing) على توزيع الطلبات القادمة من المستخدمين على عدة سيرفرات بهدف تحسين الأداء ويتم ذلك عبر مُوازن حمل (Load Balancer) يقوم بتوجيه الطلبات إلى السيرفرات وفق آليات متعددة : من أشهر هذه الآليات:

- التوزيع الدائري (Round Robin) : حيث يتم توزيع الطلبات بالتساوي وبالتتابع على السيرفرات، أي يتم إرسال الطلب الأول إلى السيرفر الأول، والطلب الثاني إلى السيرفر الثاني، وهكذا بشكل دوري.
- التوزيع العشوائي (Random Distribution) .
- التوزيع حسب الحمل (Least Load / Least Connections) : حيث يتم توجيه الطلب إلى السيرفر الأقل انشغالاً أو الأقل استقبالاً للطلبات في تلك اللحظة.

من سليات Load Balancing:

بعد قيام مُوازن الحمل (Load Balancer) بتوجيه طلب المستخدم إلى أحد السيرفرات، فإنه لا يحتفظ عادةً بأيّة معلومات عن حالة المُستخدم أو عن الـ Session الخاصة به، بل يُصبح التواصل مُباشراً بين المُستخدم والسيرفر الذي استلم الطلب.

لذلك في حال توقّف هذا السيرفر أو حدوث عطل فيه، قد يتم فقدان الـ Session والبيانات المُرتبطة بها، مما يُجبر المُستخدم على إعادة بدء العمل أو تسجيل الدخول من جديد.



■ العُنقدة (Clustering):

تعتمد العُنقدة على تشغيل عدّة سيرفرات تعمل معاً كمجموعة واحدة بهدف تحقيق الاعتمادية العالية (High Availability) وتحمل الأعطال (Fault Tolerance).
هنا لا تكون الـ Sessions مُخزّنة داخل سيرفر واحد فقط، وإنما يتم مشاركتها بين السيرفرات، إما عبر تخزين مركزي مشترك أو عبر مزامنة الـ Sessions بين جميع السيرفرات داخل الـ Cluster.
لذلك إذا توقّف أحد السيرفرات أثناء عمل المستخدم، يمكن تحويل الطلبات إلى سيرفر آخر داخل الـ Cluster دون فقدان الـ Session أو البيانات المرتبطة بها، وغالباً دون أن يشعر المُستخدم بحدوث العطل أو انقطاع الخدمة.

فائدة اربيسيزية :

التسامح مع الأخطاء (Fault Tolerance):

هي قدرة النظام على الاستمرار في العمل رغم حدوث خطأ، أي أنه لا يمكن منع الأعطال من الحدوث لكن يمكن التعامل معها بحيث لا يؤدي الخطأ إلى انهيار النظام كاملاً بل يتأثر الجزء المحدد بالعطل فقط.
- نرى هذه التقنية مُطبّقة بشكل واضح في Clustering، Load Balancing فعند تعطل server يتم توقيفه والانتقال لـ Server آخر دون إيقاف النظام كاملاً بغض النظر عن ضياع الـ data.

10. الإتاحة (Availability):

- تعني كم من الوقت يستطيع البرنامج أن يعمل دون توقف.
- $$Availability = time\ to\ fail / (time\ to\ repair + time\ to\ fail)$$
- time to fail: الزمن بين توقّفين، أي كم من الوقت استغرق ليتعطل مرة أخرى بعد العطل الأول.
 - time to repair: زمن الإصلاح، أي كم من الوقت استغرق لإصلاح العطل الحاصل.
- في حال كان الزمن بين توقفين كبير (5 دقائق مثلاً) وزمن الإصلاح صغير (10ms مثلاً) عندها ستكون قيمة الإتاحة رقم قريباً من 1 (القيمة العظمى للإتاحة هي 1)، هذه الحالة هي أفضل حالة .
- ◀ كلما كَبُرَ زَمَنُ التَّوَقُّفِ وصَغُرَ زَمَنُ الإصلاح كلما اقتربنا من الحالة المثالية للإتاحة.
- إذا كان النظام يحتاج إلى مدة طويلة نسبياً للإقلاع والتشغيل، كأن يستغرق دقيقة كاملة للبدء بالعمل بينما يتعرّض لأعطال مُتكررة كل دقيقتين، فلن يتمكن من تقديم خدمة مستقرة للمستخدمين، لأن جزءاً كبيراً من الوقت سيُستهلك في إعادة التشغيل والاستعادة بدلاً من تقديم الخدمة الفعلية، لذلك تكون قيمة الإتاحة (Availability) في هذه الحالة منخفضة جداً أو شبه معدومة.
 - ◀ ولهذا يجب أن يكون زمن الإصلاح صغيراً جداً، لأن أي توقف طويل يؤدي إلى انخفاض إتاحة النظام.
 - كما أن الأعطال لا تقتصر على توقّف البرنامج فقط، بل قد تشمل توقّف الخادم نفسه أو حدوث مشكلة في الشبكة أو قاعدة البيانات، لذلك لا يُفضّل الاعتماد على إعادة التشغيل اليدوية لأنها قد تستغرق وقتاً طويلاً، بل تُستخدم وسائل إصلاح سريعة مثل إعادة التشغيل التلقائي (Auto Restart)، والتحويل إلى خادم بديل (Failover)، واستخدام النسخ المتماثلة (Replication)، إضافةً إلى أنظمة المراقبة والتنبيه لاكتشاف الأعطال بسرعة وتقليل زمن التوقف.

ومن كلامنا السابق نستنتج:

نقول عن برنامج أن إتاحتته عالية عندما يكون إما زمن التوقف أكبر ما يمكن أو زمن الإصلاح أصغر ما يمكن للحصول على أقرب قيمة للـ 1.

في الحياة العملية الواقعية تكون غالبية البرامج زمن إصلاحها كبير، لذلك يحاول المسؤولون عنها جعلها قليلة أو معدومة التوقف قدر الإمكان.

بعد أن تعرّفنا على مفهوم الإتاحة بشكل عام لتتعرف عليها من وجهات نظر مختلفة:**■ (1) من وجهة نظر الجهة الطالبة:**

- يجب ألا يتوقف النظام أكثر من ساعة في اليوم أو في السنة أو لا يتوقف أبداً.
- يجب ألا يتوقف النظام في أوقات معينة مثل أوقات الدوام الرسمي.
- يتم تحديد أمور الإتاحة بالتفصيل قبل البدء بالتنفيذ.

■ (2) من وجهة نظر المصمم (التنفيذ):

كيف يمكن للمُنفِّذ أن يضمن الإتاحة أثناء التنفيذ؟

من خلال التسامح مع الأخطاء (Fault Tolerance) الذي تكلمنا عنه سابقاً، حيث يتم تطبيقه في:

Thread Pool : في حال حدوث عطل مع مُستخدم لا يتوقف النظام بل يتوقف الـ Thread المسؤول عن خدمة هذا المُستخدم فقط.

Clustering & Load Balancing : في حال تعطل أحد السيرفرات يتم توزيع عمله على المخدمات الباقية، حتى في حال تعطل السيرفر المركزي فيمكن فوراً استبداله بالبدايل الجاهزة.

- إذا كانت المشكلة Software سيتوقف Thread واحد
- وإذا كانت المشكلة Hardware سيتوقف Server واحد دون التأثير على كل المنظومة البرمجية والتوقف عن العمل.

11. الاستعادة من الأعطال (Fail Over):

عند حدوث عطل كامل في النظام، مثل توقف جميع السيرفرات داخل الـ Data Center، يجب أن يكون النظام قادراً على الإقلاع والعمل من جديد بشكل صحيح بعد توقّفه نهائياً.

إنّ نجاح عملية الإقلاع لا يعتمد فقط على إعادة تشغيل السيرفرات، بل أيضاً على سلامة واتساق البيانات (Data Consistency) ففي بعض الحالات قد تتم إعادة تشغيل السيرفرات دون أن يتمكن النظام من العمل بشكل صحيح، والسبب لا يكون في الخادم نفسه وإنما في البيانات الموجودة داخل قاعدة البيانات، لأن النظام أثناء الإقلاع يعتمد على هذه البيانات والإعدادات لكي يبدأ العمل. لذلك يجب أن تكون البيانات سليمة ومتناسقة (Coherent)، إذ إن وجود أخطاء أو عدم تناسق في البيانات نتيجة انقطاع مفاجئ أو عملية غير مكتملة قد يؤدي إلى فشل بعض الخدمات أو حتى تعذر إقلاع النظام بالكامل.

- **زمن إصلاح العطل:** كم الزمن اللازم لإعادة تشغيل البرنامج والقيام بعملية backup للـ DB والزمن اللازم لإعادة تشغيل الـ application server عند حدوث عطل معيّن.



■ يُؤثر على الـ Failover عوامل عديدة:

- عدد المخدمات
- استعمال ACID بشكل صحيح
- القيام بـ Backup يومي
- وجود Replication على مستوى الـ Data Base.

■ DB Replication:

- هي أحد أدوات الـ Fail Over تستخدم للحفاظ على استمرارية عمل النظام وتقليل احتمال فقدان البيانات أو توقف الخدمة عند حدوث الأعطال. وتعتمد فكرتها على إنشاء نسخة أو أكثر من قاعدة البيانات الأساسية بشكل مستمر، بحيث يُمكن الانتقال إلى النسخة الأخرى في حال حدوث مشكلة في القاعدة الأساسية، مثل انقطاع الكهرباء أو توقف أحد السيرفرات.
- والهدف من ذلك ليس فقط الحفاظ على البيانات، وإنما أيضاً الحفاظ على سلامة وتناسق البيانات (Data Coherence) ، لأن وجود قاعدة بيانات تحتوي على أخطاء أو بيانات غير متناسقة قد يمنع النظام من الإقلاع والعمل بشكل صحيح.
- كما يتم الاعتماد أيضاً على الـ Daily Backup، والذي يُعتبر في بعض الحالات أكثر أماناً من الـ Mirror (Replication) المباشر:
- لأن النسخ المتزامنة قد تنتقل الأخطاء أو البيانات غير المتناسقة مباشرة إلى جميع النسخ أما الـ Backup اليومي فيوفر نسخة ثابتة وسليمة يمكن العودة إليها لاحقاً، حتى لو أدى ذلك إلى خسارة جزء من البيانات الحديثة، لكنه يضمن أن البيانات المخزنة متناسقة وقابلة للاستعادة بشكل صحيح.
- أحياناً نلجأ لإعادة إقلاع النظام يدوياً عبر إحضار مجموعة من المهندسين لهم معرفة جيدة بالـ Business ليقوموا بتحليل المعطيات والبيانات وتحديد المتضرر منها وإصلاحه سواء يدوياً أو آلياً.

12. سهولة الاستخدام (Usability):

يبدأ التفكير بسهولة الاستخدام أثناء وضع المتطلبات وذلك بتحديد:

■ المستوى التعليمي للمستخدم النهائي:

هل يملك المستخدم خبرة في التعامل مع البرامج؟

■ الزمن اللازم لإدخال معطيات محددة وتكرار الإدخال:

في التطبيقات المؤسسية لا يُمكن جعل عملية إدخال البيانات معقدة أو طويلة، خاصة عند المستخدمين الذين يتعاملون مع عدد كبير من عمليات الإدخال بشكل يومي، مثل المحاسبين وموظفي الإدخال. لذلك يجب تصميم واجهات الإدخال بطريقة تقلل الوقت والجهد المطلوبين، وتجنب كثرة خطوات التنقل غير الضرورية، مع تجميع الحقول المرتبطة ضمن واجهات واضحة وسريعة الاستخدام .

◀ أي إذا كانت البيانات مترابطة وبسيطة، يُفضل تقليل عدد خطوات الإدخال وعدم تقسيمها إلى صفحات كثيرة، لأن كثرة التنقل بين الصفحات قد تزيد الوقت اللازم لإنجاز العمل.

■ أنواع التقارير المطلوبة:

عند زيادة أهمية التقرير أو حساسية المعلومات الموجودة فيه، تصبح عملية التحقق والتأكد أكثر أهمية، لذلك قد تتطلب بعض العمليات إضافة خطوات تحقق أو تأكيد إضافية لضمان صحة البيانات وتقليل الأخطاء.

■ التنبيهات والتحذيرات:

يجب تنبيه المُستخدم للأمر الضرورية مثل محاولة اختراق حسابه. بينما لا يجب إشغاله بتنبيهات لأمر قليلة الأهمية.

■ زمن التدريب المناسب لطبيعة عمل المؤسسة.

■ سهولة الإلغاء والتعديل.

يختلف مفهوم سهولة الاستخدام (Usability) بحسب طبيعة البرنامج ونوع العمل الذي يقدمه :

لا يوجد تصميم واحد مناسب لجميع التطبيقات. فبعض الأنظمة تحتاج إلى الدقة والتحقق أكثر من السرعة، بينما تحتاج أنظمة أخرى إلى السرعة وسهولة التفاعل حتى لو زادت احتمالية بعض الأخطاء البسيطة.

فعلى سبيل المثال:

1. في الأنظمة المصرفية :

تكون العمليات المالية حساسة جداً، لذلك لا يكون الهدف تنفيذ العملية بسرعة، وإنما التأكد من صحة البيانات بشكل كامل. ولهذا السبب تقسم العملية إلى عدة مراحل تتضمن الإدخال والتدقيق والتأكد قبل التنفيذ، بهدف تقليل الأخطاء وتحقيق مستوى أعلى من الأمان والموثوقية.

مثال:

عملية تحويل مبلغ كبير:

1. إدخال البيانات

2. مراجعة المعلومات

3. تأكيد العملية



2. أما في تطبيقات الشبكات الاجتماعية:

غالباً تكون السرعة وسهولة التفاعل أهم من وجود خطوات تحقق كثيرة، لذلك يتم تصميم الواجهات بطريقة تسمح للمستخدم بتنفيذ العمليات بسرعة وبأقل عدد ممكن من الخطوات، مع تقبل احتمال أكبر لحدوث بعض أخطاء الإدخال البسيطة.

إذاً:

- في الأنظمة الحساسة ◀ نركز على الدقة والتحقق حتى لو زاد عدد الخطوات .
- في الأنظمة التكرارية السريعة ◀ نركز على السرعة وتقليل خطوات الإدخال.

Don't watch the clock ؛ do what it does. Keep going

13. إمكانية الوصول (Accessibility):

وهي القدرة على الوصول للمعلومة والاستفادة من البرنامج حتى لو كان الشخص يُعاني من مشاكل معينة في النظر أو السمع أو الحركة أو غيرها. مثل:



- وجود المُكَبِّر (Magnifier) في نظام تشغيل Windows لضعاف النظر
- أو إمكانية تكبير الصفحة (Zoom).
- إمكانية تثبيت زر Shift لمن لا يمكنه استعمال كلتا يديه.
- دعم قارئ الشاشة لحالات الضعف الشديد بالنظر.

تحدد هذه الميزات حسب الفئة المُستهدفة من النظام، فمن أجل كل حالة هناك مجموعة حلول وتسهيلات يمكن إضافتها لمساعدة أي شخص يُعاني من هذه الحالة على استخدام البرنامج والاستفادة منه قدر الإمكان.

14. التنافس على الموارد (Concurrent Access):

تظهر مُشكلة التنافس على الموارد عندما تحاول عدّة عمليات (Processes) أو عدّة Threads الوصول إلى نفس المورد أو البيانات في الوقت نفسه، لأن موارد النظام مثل المُعالج والذاكرة وقواعد البيانات تكون محدودة دائماً مقارنةً بعدد العمليات المُتزامنة حيث أن زيادة الموارد تُعتبر مُكلفة جداً. وينتج هذا التنافس بين العمليات أو بين الـ Threads عند محاولة الوصول إلى نفس المورد المشترك، مما قد يؤدي إلى حدوث تعارض أو أخطاء في البيانات.

ولمعالجة هذه المُشكلة توجد عدة حلول من أهمها :

1. استخدام الـ Synchronization :

حيث يتم تنظيم وصول العمليات إلى المورد المشترك بحيث تُنفذ الطلبات بشكل مُرتب ومُتتابع، كأن يتم وضعها ضمن طابور انتظار، بحيث لا تستخدم أكثر من عملية المورد نفسه في اللحظة نفسها. لكن الـ Synchronization تُعتبر عملية مُكلفة وتحتاج وقتاً إضافياً للتنفيذ، خاصةً في التطبيقات المؤسسية الكبيرة التي تحتوي على عدد كبير من المستخدمين والطلبات المتزامنة، لذلك يُصبح استخدامها بشكل واسع أمراً صعباً لأنه قد يخفف من الأداء ويقلل من سرعة التنفيذ.

2. تسمح بعض الأنظمة الحديثة بوجود قدر معين من التنافس والعمل المتوازي للحصول على سرعة وأداء أفضل، مع الاعتماد على خصائص قواعد البيانات مثل ACID للمحافظة على صحة البيانات وتقليل الأخطاء.

كما تعتمد كثير من التطبيقات المؤسسية الحديثة على المُعالجة غير المتزامنة (Asynchronization) بهدف تحسين الأداء والسماح بتنفيذ عدد أكبر من العمليات بشكل متوازي دون إيقاف النظام أو تعطيل المستخدمين الآخرين.

من أشهر أمثلة التنافس : هو الركاب في محطة القطار.

أما تَقْنِيَةً فالمثال الأشهر هو مثال ATM (ماكينة سحب الأموال):

يطلب الزوج 100 ◀ تتحقق الآلة من توفّر المبلغ (500) ◀ تسحب الآلة المبلغ (100) ◀ تُقدِّمهُ للزوج.
تطلب الزوجة 100 ◀ تتحقق الآلة من توفر المبلغ (500) ◀ تسحب الآلة المبلغ (100) ◀ تُقدِّمهُ للزوجة.

يقوم الزوج والزوجة كلاهما بسحب مبلغ 100 من نفس الحساب وفي نفس اللحظة أي على التوازي فيحصل تنافس ليتم تنفيذ أحدهما قبل الآخر (تم شرح آلية عمل المثال في المحاضرة السابقة بالتفصيل).

حل المشكلة

والآن بعد الانتهاء من توصيف جميع المشاكل والإجابة على سؤال "ما هي المشكلة"، سنتنقل لاكتشاف الحلول والإجابة عن سؤال "كيف يتم حل المشكلة".

البُنيان (Architecture):

عندما كنا أطفالاً صغاراً كان من أكثر الأشياء إبهاراً لنا هو التلفاز! كيف تم صنعه؟ من هو الشخص العبقرى الذي ابتكره؟ إلى أن كبرنا واكتشفنا أنه عبارة عن صناعة تجميعية، يجتمع عددٌ أشخاص بخبرات مختلفة لصنع أقسام التلفاز ثم تجميعها معاً لتعطي المنتج النهائي. لكن أصبح السؤال الجديد: كيف تمكّن هؤلاء الأشخاص من العمل بطريقة متكاملة إلى هذا الحد؟ كيف تمكّنوا من إخراج منتج مترابط بهذه الطريقة؟

الحل يكمن في البُنيان! أي المعرفة المُسبقة بالنموذج الذي ستحصل عليه، لذلك كل شخص يعرف مهمته وما شكلها النهائي وكيف سيتم ربطها ومكاملتها مع باقي الأجزاء.

- البُنيان هو الذي سيقدر كيف سيتم العمل وكيف ستتم مكاملة الأجزاء مع بعضها وذلك التحديد يتم قبل البدء بالعمل لتجنب حالات عدم التطابق عند التجميع.

قبل البدء بالعمل يُبنى تصوّر مستقبلي وخطة واضحة يلتزم بها كل الفريق.

الالتزام بالبنيان هو أكبر ضمان لصحة العمل على التوازي.

مُهمّة البُنيان هي فصل المتطلبات ضمن مجموعة من نماذج التصميم (أي فصل كل جزء لوحده بشكل مستقل) مع ضمان القدرة على التكامل الصحيح لاحقاً.

عند فصل الأقسام بشكل مستقل وفي حال عدم إتمام أحد الأقسام بالكفاءة المطلوبة فلن يؤدي إلى انهيار المنظومة كاملة بل سيؤثر على هذا القسم وحده، وهذا أهم شيء في البُنيان.

يسمح البُنيان الجيّد بتطوير برمجيات ذات نوعية مستقلة نسبياً عن نوعية المبرمجين، لنشر أكثر:

كما ذكرنا مهمة البُنيان الأساسية هي فصل المتطلبات (الوظيفية عن غير الوظيفية والوظيفية عن الوظيفية وغير الوظيفية عن غير الوظيفية) ووضعها ضمن مجموعة من نماذج التصميم، هذا الفصل يجعل العمل أكثر أماناً وأقل حجماً (من يعمل على Security لا دخل له بالـ Business ومن يعمل على Tracing لا دخل له بالـ Load Balancing, Clustering). بهذه الطريقة إذا كان أحد المسؤولين عن Security ذو كفاءة منخفضة نسبياً فلن يتأثر التطبيق أي لن يكون البرنامج سيئاً لأن فيه مبرمجين سيئين، فالمُبرمج السيئ لا يؤثر سوى على قسمه فقط وليس على البرنامج ككل.

التَفَرُّعُ يُمْكِنُ تَجْمِيعُهُ وَلَكِنِ التَّوَازِي لَا يَلْتَقِي إِلَّا فِي

الانتهائية...

فائدة:

البنّان هو أول عملية في دورة التصميم وتليها مرحلة التحليل ويحتاج البنّان إلى شخص ذي كفاءة وخبرة عالية للقيام بتصميم هذا البنّان، هذه المرحلة هي حجر الأساس لإنجاز المنظومة البرمجية كاملة، كما يقوم هذا الشخص بمتابعة التنفيذ لاحقاً للتأكد من سير العملية كما خطط لها في البنّان.

في حال حدث عطل في النظام فهو مسؤولية المهندس المسؤول عن البنّان والتصميم والتحليل وليست مسؤولية المبرمج، فالمهندس هو المسؤول عن وضع تصوّر للخروج النهائي وهو المسؤول عن مراجعة خرج كل مهمة منفردة لتطابق المطلوب.

مستويات البنّان :

والآن بعد التعرّف على البنّان وأهميته وفهماته، لتعرّف على مستويات البنّان السبعة:

- أي برمجة تدعم العمل على التوازي، بحاجة إلى هذه الطبقات كلها.
- في الوقت الحالي يبدأ البنّان كأخفض مستوى من لغات البرمجة OOP، لكن قبل البدء بالحديث عن الطبقات الحالية سنتكلم قليلاً عن الطبقات الحقيقية والتي كانت مهمة جداً سابقاً وأصبحت من المسلّمات في أيامنا هذه.
- أخفض مستوى بالبنّان فعلياً هو بنّان الحاسوب (العتاديات)، حيث قديماً كان الـ Keyboard (لوحة المفاتيح) تحوي الـ RAM والـ CPU تحت الأزرار كأنه حاسب كامل، لكن السيئ فيه أن أقصى حجم ممكن للـ RAM هو 64kB (64 كيلو بايت) فقط.
- لذلك لاحقاً تم تطبيق مفهوم البنّان وفصل المكونات عن بعضها، ثم ظهر Assembler ثم لغات البرمجة، ومع تقدّم الزمن أصبحنا قادرين على زيادة حجم RAM كما نشاء دون الخوف من نفاذ الـ 64kB المتاحين والاضطرار للكتابة على Extended Memory، وكل هذا أثناء التفكير بالكود! كما أصبحنا قادرين على وضع عدة معالجات وبعده أنوية فأصبح بنّان الحواسيب من الماضي.
- في الوقت الحالي تبدأ مستويات البنّان كأدنى مستوى من:

1. لغات البرمجة (Object Oriented Programming (OOP):

أي كمبرمج في أيامنا هذه عليك الاهتمام بالبرمجة فقط ولا علاقة لك بالـ RAM وكيفية التخزين، بينما كانت مهمة مهندسي الماضي الأساسية هي معالجة مثل هذه الأمور البنيوية.

ما فائدة OOP وماذا تقدّم؟

تقدّم OOP خمس خصائص:

■ (1) التغليف (Encapsulation):

يُقصد بها إخفاء التفاصيل الداخلية للكائن (object) وإتاحة الوصول لها عبر دوال محددة له ، وهدفها حماية الكود من التعديلات الخارجية وجعله قابلاً للتعديل بشكل محلي Locally فقط للبنية الداخلية الخاصة بالـ classes, objects ، أي الفصل بين البنية الداخلية والمستخدمين.

■ (2) الوراثة (Inheritance):

يُقصد بها إنشاء class جديد يرث خصائص وسلوك class قديم (أب). هدفها تقليل التكرار وإعادة استخدام الكود (Reusability).

■ (3) البرمجة التزايدية بالوراثة (Incremental Inheritance):

يُقصد بها بناء النظام بشكل تدريجي عبر سلسلة من الوراثة بحيث كل class يُضيف خدمات وميزات جديدة دون التعديل على الخدمات القديمة. هدفها تكبير النظام خطوة بخطوة دون التأثير على الخطوات القديمة.

■ (4) تعدد الأشكال (Polymorphism):

بناء Classes يُمكن أن تأخذ عدّة أشكال وتحوي دوال يمكنها التصرف بطرق مختلفة حسب الكائن، مثال: إذا كان لدينا 2 classes أحدها للطالب والآخر للدكتور، الطالب بعد تخرجه تابع دراسته وأصبح دكتوراً فسيصبح له class ثاني وتنتقل معلوماته وهذا خطأ،
 التصرف الصحيح إنشاء class واحد اسمه human يأخذ أدوار مختلفة بدورة حياة البرنامج، أي الطالب والدكتور كلاهما human لكن الفرق بينهم زمني.

■ (5) الربط المتأخر (Late Binding):

يُقصد به تحديد أي دالة سيتم تنفيذها أثناء التشغيل (at Runtime) وليس أثناء الترجمة (Compile)، حيث تشير الدوال إلى الـ body الخاص بها at Runtime بشكل متأخر، أي يمكن إضافة classes جديدة على البرنامج at Runtime دون الحاجة للقيام بعملية Compile.
 حيث لغات البرمجة المختلفة لكل منها features وتصميم مميز وليس فقط Syntax مختلف.

فائدة:

- في الأنظمة القديمة لم يكن الحاسب يتعرّف على الأجهزة بشكل تلقائي، خاصةً عند استخدام منافذ مثل Serial Port و Parallel Port، لذلك كان المُستخدم يحتاج إلى تثبيت تعريفات وإجراء إعدادات يدوية حتى يتمكن النظام من التواصل مع الجهاز ونقل البيانات بشكل صحيح.
- أما الآن أصبح توصيل الأجهزة الخارجية أسهل بكثير بسبب وجود Bus ودعم ميزة Plug and Play فعند توصيل جهاز مثل الفأرة أو الطابعة عبر منفذ USB، يقوم نظام التشغيل بالتعرّف عليه تلقائياً ويصبح جاهزاً للاستخدام مباشرة دون الحاجة إلى إعدادات معقدة.

2. أنماط التصميم (Design Patterns):

هي حلول عامة ومُجربة لمشاكل متكررة نواجهها بشكل مستمر في عملية تصميم وتطوير البرمجيات. هذه الأنماط هي مناهج وطرق تنظيمية تقدّم إرشادات للمُبرمجين حول كيفية هيكلة الكود لحل مشكلة معينة بطريقة فعّالة ومُثلى. الهدف منها هو توفير لغة مشتركة للمُطوّرين وتجنب الوقوع في نفس الأخطاء التصميمية مراراً وتكراراً.

أمثلة على أنماط التصميم:

توجد العديد من أنماط التصميم المُستخدمة على نطاق واسع، ومن أبرزها:

■ MVC (Model-View-Controller)

هذا النموذج يفصل بين ثلاثة مكونات رئيسية:

- Model: تُمَثِّل البيانات ومنطق العمل.
- View : تُمَثِّل واجهة المستخدم
- Controller : يتعامل مع مدخلات المستخدم ويُحدِّث النموذج والواجهة بناءً عليها.

■ ORM (Object-Relational Mapping)

تقنية تُمكن المُطَوِّرِينَ من التفاعل مع قواعد البيانات العلائقية باستخدام كائنات برمجية (Objects) بدلاً من كتابة استعلامات SQL مباشرة.

■ DAO (Data Access Object)

نمط يهدف إلى تجريد كيفية الوصول إلى البيانات، مما يفصل منطق الوصول إلى البيانات عن منطق العمل الأساسي للتطبيق.

■ AOP (Aspect Oriented Programming)

وقد تحدثنا عنها سابقاً.

فوائد استخدام أنماط التصميم:

لكل نمط تصميم مميزات وفوائد خاصة به.

على سبيل المثال نمط MVC يقدم ميزة هامة جداً:

- مرونة في تغيير الواجهة :

يسمح بتغيير واجهة المُستخدم (View) بشكل كبير دون الحاجة إلى إعادة كتابة أو تعديل منطق العمل الأساسي Model و Controller .

- دعم منصات متعددة:

يُمكن تشغيل نفس منطق العمل على منصات مختلفة مثل الويب أو تطبيقات الموبايل بسهولة، دون الحاجة إلى إعادة بناء البرنامج بالكامل لكل منصة. هذا يُوفِّر الكثير من الوقت والجهد على المُطَوِّرِينَ.

إذاً :

كل نمط من أنماط التصميم تم تطويره ليناسب حالات استخدام محددة ويُعالج مشاكل تصميمية بعينها، مما يُساهم في بناء برمجيات أكثر قوة، مرونة، وسهولة في الصيانة والتطوير.

3. إطار العمل (Framework):

يُقصد به هيكل أو بيئة برمجية جاهزة تحتوي على أدوات وقواعد تساعد على بناء التطبيق بدقة وبسرعة وتنظيم، تقوم أطر العمل بتجميع مجموعة من أنماط التصميم (Design Patterns) المُتكاملة التي تعمل مع بعضها بشكل صحيح حتى يتمكن الشخص من الاستفادة منها جميعاً، ولكي يتجنَّب استخدام أنماط متضاربة مع بعضها أو يضيع بين الأنماط.

تتكون الأطر من مجموعة أدوات معها مجموعة توجيهات هدفها توجيه كيفية استخدام الأدوات المتوفرة للحصول على أفضل نتيجة وأفضل Software.

كل framework يتألف من جزأين:

- قسم الأدوات: technical framework
- قسم التوجيهات والإدارة: Software Engineering framework

مقارنة بين الـ Design Pattern والـ Framework :

Design Pattern	Framework
عبارة عن فكرة أو أسلوب تصميم لحل مشكلة برمجية متكررة.	عبارة عن بيئة عمل أو هيكلية جاهزة لبناء التطبيقات.
يُعتبر أقل مستوى وأكثر بساطة.	يُعتبر أعلى مستوى وأكثر شمولاً.
لا يحتوي على كود جاهز كامل، وإنما يُقدم طريقة تنظيم للحل.	يحتوي على مكونات وكود وأدوات جاهزة تساعد على تطوير التطبيق.
يُمكن للمُبرمج تطبيقه بالطريقة التي تناسبه.	يفرض هيكلية وقواعد معينة على طريقة بناء التطبيق.
يُعالج مشكلة تصميم محددة.	يُوفّر بنية متكاملة للتطبيق ويستخدم عدة Patterns داخله.
أمثلة Singleton, Factory, Observer	أمثلة Spring, .NET, Django

كما أن الـ Architectural Patterns تركز غالباً على تصميم الأنظمة الكبيرة وطبقة الـ Middleware، وتهتم بمشاكل مثل:

- التوازي (Concurrency)
- التوزيع (Distribution)
- التزامن (Synchronization)
- القابلية للتوسع (Scalability)

إذاً:

Design Pattern يُعالج مشكلة تصميم صغيرة ومحددة، بينما الـ Framework يُقدم بنية متكاملة لبناء التطبيق، وتستخدم داخله عدة Patterns معاً

The only way to do great work is to love what you do